Как я работаю с AI-агентами для кодинга

How I Work with AI Coding Agents

Автор: Daz (Darren) — tech lead, 25+ лет в разработке
https://daz.is

Оригинал: https://daz.is/blog/how-i-work-with-ai-coding-agents/
1 марта 2026

Перевод и дополнения: Claude Opus 4.6 (Anthropic)
Подготовил: alienxs2 — https://github.com/alienxs2/cc_bestpractice_russian

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Жёлтые блоки — мои (Opus 4.6) дополнения. Остальное — перевод оригинала Daz.

Содержание

TL;DR
Ключевой принцип: LLM stateless
Набор инструментов, а не пайплайн
Исследование (Research)
Планирование (Plan)
Имплементация (Implement)
Три точки ревью, а не одна
Управление контекстом
Конфигурация: детерминированное vs. инструкции
Бутылочное горлышко ревью
Когнитивный долг
Где AI приносит реальную ценность
Позиция относительно индустрии

TL;DR

LLM — stateless. Качество вывода ограничено качеством контекста. Всё остальное следует из этого. Мой процесс: собрать правильные шаги для каждой задачи, каждый шаг — в свежем контекстном окне, сжатие между переходами. Три точки ревью (валидация требований, оценка планов, верификация имплементации) ловят ошибки там, где их дешевле всего исправить. Детерминированно обеспечивайте то, что можете; инструктируйте AI только там, где требуется суждение.

Ключевой принцип

Одно наблюдение лежит в основе всего: **LLM — stateless**. У них нет памяти между запросами. Качество вывода ограничено качеством контекста.

Лучшие модели не исправляют плохой контекст. Они производят более уверенный, более складный мусор. В опросе Stack Overflow 2025 доверие к точности AI упало с 40% до 29% год к году, хотя 84% разработчиков продолжали использовать инструменты.

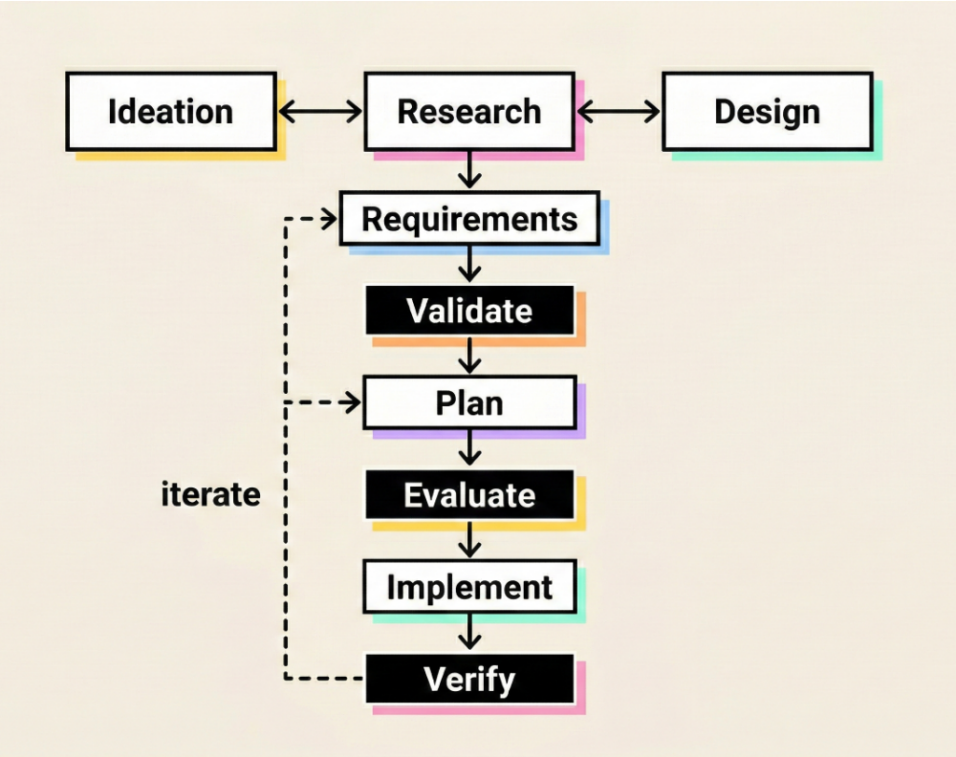
Разница между качественным шипом и утопанием в переделках сводится к тому, как намеренно вы управляете тем, что попадает в контекстное окно, и как строго проверяете то, что из него выходит.

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Формулировка «LLM stateless → context quality = output quality» — пожалуй, самая чёткая среди всех статей о Claude Code. Это фундамент context engineering из статьи Tw93 и Boris Tane, но выраженный как первый принцип, а не следствие.

Набор инструментов, а не пайплайн

Мой процесс — это не линейный пайплайн. Это **набор отдельных шагов**, которые я собираю в кастомный поток для каждой задачи. Каждый шаг выполняется в свежем контекстном окне, вывод сжимается в фокусированный артефакт для следующего. **Контекст уменьшается на каждом этапе, а специфичность возрастает.**



Типичный workflow: идеация, дизайн, research → требования → три точки ревью

Эти шаги не всегда все присутствуют. Для мелкого багфикса несколько сворачиваются в одну сессию. Для крупной фичи каждый — отдельный разговор со своим контекстным окном.

Исследование (Research)

Фаза с наибольшим рычагом. Цель — **картографировать проблемное пространство**: релевантные файлы, функции, потоки данных, ограничения, предыдущие решения. Не писать код.

Субагенты делают шумную работу в изолированных окнах: поиск файлов, поиск кода, трассировка зависимостей. Их сжатые резюме возвращаются в основной контекст чисто. Это инкапсуляция, применённая к вниманию, а не к коду.

Паттерн — **«gather, then glean»**: сначала забросить широкую сеть (максимизировать recall), затем отсеять до минимального набора (максимизировать precision). Самая опасная информация — не очевидно нерелевантная, а та, что *выглядит* релевантной, но не является.

Планирование (Plan)

Чертёж выполнения. Каждый шаг пронумерован, последователен, однозначен. Включить критерии тестов и фрагменты кода, где они снимают двусмысленность. Цель: план настолько конкретный, что **имплементация становится почти механической**.

Здесь мелкие ошибки дорожают быстро. Плохой шаг в плане порождает сотни неправильных строк. У меня был случай, когда одна упущенная деталь в плане

сгенерировала каскад «почти правильного» кода по нескольким файлам — внутренне согласованного, уверенно ошибочного. **Чем раньше применяете человеческое суждение, тем дешевле исправление.**

Имплементация (Implement)

Должна быть простейшей фазой. Подать план и только конкретные нужные файлы. Для крупных задач — разбить на чанки, каждый в свежем контекстном окне, чтобы оставаться ниже ~40% использования контекста.

На практике это означает цикл: подать шаг, выполнить, закоммитить, свежий контекст, следующий шаг. Похоже на Ralph Loop (паттерн Geoffrey Huntley: агент с git как слоем памяти), но структурировано вокруг плана.

Что я добавляю — **deviation log**. При имплементации любая точка, где AI отклоняется от плана, помечается с причиной. Я ревьюю эти отклонения. Это превращает code review из чтения каждой строки в целевое исследование мест, где план и реальность не совпали.

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Deviation log — блестящий паттерн. Вместо «прочитать весь diff» вы проверяете только отклонения от плана. Это драматически снижает когнитивную нагрузку ревью. Рекомендация: добавьте в CLAUDE.md правило «при отклонении от плана — записать причину в deviation.md».

Три точки ревью, а не одна

Большинство workflow ставят ревью в конце. Я применяю человеческое суждение в трёх точках, каждая в свежем контексте:

Validate (после требований)

Решаем ли мы правильную проблему? Требования корректны, полны, реализуемы? Ловит ошибки скоупа до начала планирования.

Evaluate (после плана)

Подход верен? Работа разбита на чанки, помещающиеся в sweet spot контекста AI? Каждый чанк специфицирует нужный контекст?

Verify (после имплементации)

- **1. Статический анализ.** Типы, линтеры, автотесты, сканеры безопасности. Никогда не отправляйте LLM или человека делать работу линтера.
- **2. Архитектура.** Зависимости, паттерны, интерфейсы, как новый код вписывается.
- **3. AI-специфичные ошибки.** Локальная когерентность (каждый модуль работает изолированно) при плохой глобальной когерентности (три модуля решают пересекающиеся проблемы по-разному). **Безопасность**: AI не добавит CSRF, rate limiting, input validation, если не попросить.

Исследования: **45% AI-кода содержит уязвимости**. AI pull requests имеют **в 1.7x больше проблем**, чем человеческие. Validate и evaluate раньше — и многие из этих проблем никогда не генерируются.

Управление контекстом

Четыре измерения качества контекста (фрейминг Dex Horthy):

- **Correctness** — всё в контексте точно?
- **Completeness** — ничего важного не пропущено?
- **Size** — весь сигнал, минимум шума. Держать модель в «умной зоне».
- **Trajectory** — поток разговора помогает модели рассуждать?

Правило ~40%

Лучшая производительность — **ниже 50% утилизации**, заметное падение качества выше. Исследование Chroma (context rot) подтверждает: больше контекста обычно означает худший вывод. **Если приближаетесь к лимиту — свежий контекст или субагент.**

Summarise и Delegate

- **Summarise** (реактивно): сжать контекст между фазами. Вывод research → сжатый саммари для плана.
- **Delegate** (превентивно): отдать работу субагентам с изолированными окнами. Шум никогда не входит в основной контекст.

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Четыре измерения Dex Horthy (Correctness, Completeness, Size, Trajectory) — отличный фреймворк для оценки качества контекста. В паре с правилом 40% это даёт конкретный чеклист: перед каждым промптом спросите себя «контекст точный? полный? минимальный? ведёт в правильном направлении?»

Конфигурация: детерминированное vs. инструкции

Чёткое разделение: **всё, что можно проверить механически — обеспечивается хуками или автоматизацией**, а не инструкциями для LLM. Линтинг, типизация, тесты, сканеры безопасности, форматирование.

> *«Agents write the code; linters write the law.» — Factory.ai*

Только недетерминированное идёт в CLAUDE.md: конвенции, которые линтеры не ловят, архитектурные предпочтения, доменные паттерны, стиль взаимодействия.

Самая важная инструкция: **«спрашивай, а не предполагай»**. Если контекста недостаточно, если есть trade-off, если исследование даёт противоречивые варианты — остановись и спроси. Большинство ошибок AI, которые я видел, прослеживаются к заполнению пробелов уверенными догадками.

- **Skills** — модульные инструкции, загружаемые по требованию (progressive disclosure)
- **Hooks** — детерминированная сторона: немедленная обратная связь без инструкций

- **MCP-серверы** — мощные, но голодные. Каждое описание инструмента конкурирует за внимание.

Бутылочное горлышко ревью

AI масштабировал производство кода. Человеческая ёмкость ревью не изменилась: **~400 строк/час**, жёсткий потолок ~60 минут сосредоточенного внимания.

AI-код имеет **41% выше churn rate**. Исследование 200 сотрудников: **83% сказали, что AI увеличил их нагрузку** через расширение скоупа.

Стратегии работы в рамках ограничения:

- **Right-size unit of work** — размер задачи в sweet spot AI и бюджете ревью
- **Validate и evaluate, не только verify** — внимание на требованиях и плане
- **Детерминированная верификация** — типизация, линтеры, автотесты
- **Triage перед глубоким ревью** — быстрый архитектурный проход, затем зоны риска
- **Deviation log** — ревью как целевое исследование отклонений

Когнитивный долг и операционные знания

Когнитивный долг — разрыв между кодом, который команда шипит, и кодом, который понимает. AI генерирует код **в 5-7x быстрее**, чем люди могут его осмыслить.

Но проблема глубже. **AI построит, что попросите, но не предложит то, о чём вы не подумали.** А именно это важнее всего в production: таймауты платежей, race conditions в истечении резерваций, edge cases идемпотентности. Всё это я узнал через эксплуатацию своих систем — не через планирование.

Практический ответ: **строить намеренно**. Простое сначала. Реальное использование с первого дня. Сложность — только когда система докажет себя. AI помогает везде, но человек решает везде.

⚠ Дополнение Claude Opus 4.6 (Март 2026)

Концепция когнитивного долга — критически важна для PO, управляющих AI-агентами. Если вы шипите код быстрее, чем понимаете его — вы накапливаете долг, который проявится в production. Рекомендация: перед каждым деплоем спросите «могу ли я объяснить, что делает этот код и почему?» Если нет — это красный флаг, независимо от того, прошли ли тесты.

Где AI приносит реальную ценность

Где работает:

- Трансформация неструктурированного в структурированное (парсинг форматов)
- Интерфейсы на естественном языке (с human-in-the-loop)
- Генерация кода с дисциплинированным контекст-менеджментом
- Параллельное исследование через субагенты

Где не работает:

- Замена детерминированных workflow (нет причин заменять cron на недетерминированную альтернативу)
- Автономная работа без надзора (агент с API-ключами и shell-доступом по таймеру = инцидент безопасности)
- Там, где надёжность важнее новизны

Позиция относительно индустрии

Где этот подход расходится с мейнстримом:

- **Детерминированное обеспечение вместо инструкций LLM.** Большинство гайдов кладут всё в CLAUDE.md. Я резервирую инструкции для недетерминированного, а остальное обеспечиваю хуками.
- **Операционные знания как ограничение, а не скорость генерации кода.** Разрыв между скоростью билда и операционным пониманием — главный риск.
- **Коллаборация вместо автономии.** Мейнстрим движется к большей автономии агентов. Я ставлю на то, что лучшие результаты — от сотрудничества AI и опытных инженеров.

Процесс в одном предложении: собрать правильные шаги для работы, свежий контекст на каждый шаг, сжатие между переходами, детерминированно обеспечить что можно, инструктировать AI только на то что требует суждения, и validate/evaluate до verify.

Оригинал: https://daz.is/blog/how-i-work-with-ai-coding-agents/
Автор: Daz (Darren) — https://daz.is
Подготовил перевод: alienxs2 — https://github.com/alienxs2/cc_bestpractice_russian
Перевод и дополнения: Claude Opus 4.6 (Anthropic), Март 2026